# Operating Systems 2022

**U3680**

**National Taipei University**

---

**Midterm Exam [Chapter 1 - Chapter 5]**

---

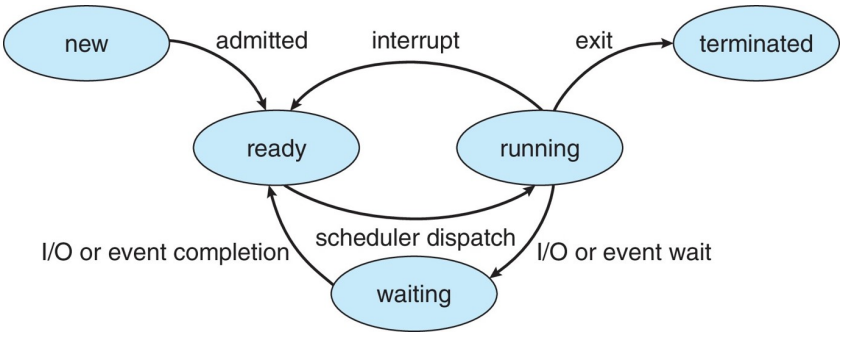
1. What are the goals of multi-programming and time-sharing? (6%)
   - multi-programming: increase CPU utilization and keep CPU busy (3%)
   - time-sharing: provide interactivity and reduce response time (3%)

2. Why do we need to distinguish user-mode instructions and kernel-mode instructions? (3%) Describe how an OS implements the kernel-mode instructions? (3%)
   - for protecting system resource and preventing a process from causing harm to other processes (3%)
   - by using a mode bit (1%) and system calls (2%)

3. What are differences between system calls and API? (8%)
   - system calls: the programming interface to the services provided by the OS; generally avialable as assembly-language instructions; each system call is a very short program focusing on a specific task (4%)
   - API: an encapsulation of system calls for programmer; usually implemented by high-level language; provide portability; can combine zero or multiple system calls to do a more complex task (4%)

4. What are the advantages and disadvantages of the micro-kernel approach to system design? (6%)
   - advantages: safer, less possible to crash, modulation, easy to extend, easier for porting (4%)
   - disadvantage: slow (2%)

5. Draw the diagram of process state and briefly describe the meaning of each state. (10%)
   - diagram of process state (5%)



- – new: the process is being created (1%)
  – ready: the process is in the memory waiting to be assigned to a processor (1%)
  – running: the process whose instructions are being executed by CPU (1%)
  – waiting: the process is waiting for events (e.g., I/O) to occur (1%)
  – terminated: the process has finished execution (1%)

6. What are the two inter-process communication models and how do they work? (10%)

   - message passing (2%): build connection using send/recv message under the assistance from OS; the sender first copies the message or data to OS memory, then the receiver copies the message or data from OS's memory to its own memory space (3%)
   - shared memory (2%): OS first creates a shared memory space for communication, then the processes can directly access the shared memory; programmer needs to take the data synchronization on their own (3%)

7. **Including** the initial parent process, how many processes are created by the program shown below? (5%)

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int i;

    for (i = 0; i < 5; i++)
       fork();

    return 0;
}
```

   there are 32 processes. (5%)

8. Which of the following components of program state are **shared** across threads in a multi-threaded process? (4%) Describe why the rest parts could not be shared? (4%)

   (a) Register values
   (b) Heap memory
   (c) Global variables
   (d) Stack memory

   - heap and global variables can be shared (4%)
   - because each thread can run different part of the code during the execution, so we need different registers and stacks for local variables and function return values (4%)

9. What are the problems when time quantum of RR becomes too short and too long? (6%)

   - too short: the context switch occurs too frequently, resulting in high overhead (3%)
   - too long: RR will become a FCFS scheduling algorithm, if a long process comes first, other processes would wait too long (3%)

10. What is the difference between preemptive and non-preemptive scheduling? (8%)

- non-preemptive: the OS can not interrupt a process if it is under its CPU burst. CPU scheduling can only be performed when a running process is terminated or switched to the waiting state (4%)
- preemptive: the OS can interrupt a process even if it is under its CPU burst. Besides a running process is terminated or switched to the waiting state, scheduling can also be performed when context switch happens or a process is switching from waiting state to ready state (4%)

11. What are the three multi-threading mapping models for user-level threads and kernel-level threads. (6%) What are the purposes of process contention scope (PCS) and system contention scope (SCS) scheduling (6%)

    - – many-to-one (2%)
      – one-to-one (2%)
      – many-to-many (2%)
    - – PCS: thread library schedules user-level threads for mapping the kernel-level threads (3%)
      – SCS: OS schedules kernel-level threads for running on processors or cores (3%)

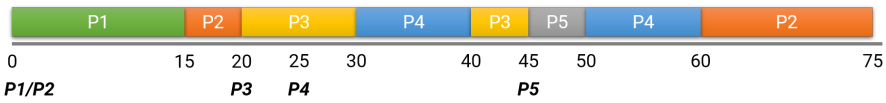
12. The following processes are being scheduled using a preemptive, priority-based, round-robin scheduling algorithm. Each process is assigned a numerical priority, with a **higher** number indicating a **higher** relative priority. The scheduler will execute the highest priority process. For processes with the same priority, a round-robin scheduler will be used with a time quantum of 10 units. If a process is preempted by a higher-priority process, the preempted process is placed at the end of the queue.

| Process | Priority | Burst | Arrival |
|---|---|---|---|
| $P_1$ | 8 | 15 | 0 |
| $P_2$ | 3 | 20 | 0 |
| $P_3$ | 4 | 15 | 20 |
| $P_4$ | 4 | 20 | 25 |
| $P_5$ | 5 | 5 | 45 |

(a) Show the scheduling order of the processes using a Gantt chart. (5%)

(b) What is the waiting time for each process? (5%)

- Gantt chart: (5%)




- waiting time:
  - $P_1$: 0
  - $P_2$: 55
  - $P_3$: 10
  - $P_4$: 15
  - $P_5$: 0

13. Using the program shown below, what are the outputs of this program? (5%)

    The outputs are: (5%, get 3% if the numbers are correct) CHILD: 0 CHILD: -2 CHILD: -8 CHILD: -18 CHILD: -32 PARENT: 0 PARENT: 2 PARENT: 4 PARENT: 6 PARENT: 8

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

#define SIZE 5

int nums[SIZE] =  {0, 2, 4, 6, 8};

int main()
{
int i;
pid_t pid;

   pid = fork();

   if (pid == 0) {
      for (i = 0; i < SIZE; i++) {
         nums[i] *= -i;
         printf("CHILD: %d ",nums[i]);
      }
   }
   else if (pid > 0) {
      wait(NULL);
      for (i = 0; i < SIZE; i++)
         printf("PARENT: %d ",nums[i]);
   }

   return 0;
}
```